CSE105/209A: Modern Algorithmic Toolbox*, PSET-3, **Deadline: 7 days from now**

Ian Chi

Instructions: Deadline in **7 days, 11:59pm**. Submit to Canvas a pdf of your solutions, with your name and ucsc id number. Feel free to discuss solutions with your colleagues (other students), explore the internet, read and be inspired from the textbooks etc. But what you type should be typed by you and you only in latex. In algorithms, we appreciate brevity and accuracy: your solutions should be to the point, often one or two sentences suffice to provide all necessary details and descriptions. Unnecessarily long answers will not get full credit.

Ex.1 (30pts) (Review of Linear Algebra)

Just to review some basic concepts from linear algebra we ask you to prove the following (if referenced, matrix A is real and symmetric):

1. Prove: If A is a real, symmetric $n \times n$ matrix, then all of its eigenvalues are real.

   solutionconsider the sesquilinear form

   $$\langle ., A. \rangle \tag{1}$$

   if we now coinsider

   $$\langle v, Av \rangle \tag{2}$$

   where v is an eigenvector of A,

   $$\begin{aligned} \lambda \langle v, v \rangle &= \langle v, Av \rangle \\ &= \langle \bar{A}v, v \rangle \\ &= \bar{\lambda} \langle v, v \rangle \end{aligned} \tag{3}$$

   thus $\bar{\lambda} = \lambda$ which implies lambda is real.

*PI: Vaggos Chatziafratis

2. Let λ_i and λ_j be two eigenvalues of a symmetric matrix A, and u_i, u_j be the corresponding eigenvectors. Prove the following: If $\lambda_i \neq \lambda_j$, then $u_i^T u_j = 0$ (inner product is zero so vectors are orthogonal).

   solution

$$\lambda_i \langle u_i, u_j \rangle = \langle Au_i, u_j \rangle = \langle u_i, Au_j \rangle = \lambda_j \langle u_i, u_j \rangle \tag{4}$$

   which can only hold if $\langle u_i, u_j \rangle$ is zero since $\lambda_i, \neq \lambda_j$

3. More generally, it is without loss of generality that we can assume that the eigenvectors of A form an orthonormal basis for $\mathbb{R}^n$ (recall, orthonormal basis is a set of vectors that are all of unit length, and any two of them are orthogonal to each other, for example the standard basis vectors form an orthonormal basis).

   Prove: Let $\lambda_1 \leq \ldots \leq \lambda_n$ be the eigenvalues of A with corresponding eigenvectors $u_1, \ldots, u_n$. Then, $A = \sum_{i=1}^n \lambda_i u_i u_i^T$ (note this $u_i u_i^T$ is **not** the inner product of the vectors and the result is a matrix).

   solution

   since the eigenvectors form an orthonormal basis we can represent any vector v as $v = \sum c_i u_i$ where u_i are the orthonormal eigenbasis vectors. since we know how each one scales, we can represent each u_i acted on by A as $\lambda_i u_i u_i^T c_i u_i = c_i \lambda_i u_i$. doing this term by term in our decomposition of v shows that A is indeed $\sum_{i=1}^n \lambda_i u_i u_i^T$

4. Prove the following characterization for the largest eigenvalue (analogous characterizations hold for all other eigenvalues): If A is an $n \times n$ real symmetric matrix, then the largest eigenvalue of A is

$$\lambda_n(A) = \max_{v \in \mathbb{R}^n \setminus \{0\}} \frac{v^T A v}{v^T v}$$

   (Hint: use the fact from part 3 that the eigenvectors form an orthonormal basis to express any vector v...)

   solution

   for any normalized vector v, we can see the equation is actually just

$$\lambda_n(A) = \max_{v \in \mathbb{R}^n \setminus \{0\}} \frac{v^T A v}{v^T v} = \max_{v \in \mathbb{R}^n \setminus \{0\}} \frac{v^T A v}{1} \tag{5}$$

   so we shall work with normalized vectors instead. any normalized vector can be written as linear combination of eigenbasis, so we can write the

term as

$$\begin{aligned}\lambda_n(A) &= \max_{v\in\mathbb{R}^n\setminus\{0\}} \left(\sum c_i w_i\right)^T A \left(\sum c_i w_i\right) \\ &= \max_{v\in\mathbb{R}^n\setminus\{0\}} \left(\sum c_i w_i\right)^T \left(\sum \lambda_i c_i w_i\right) \\ &= \max_{v\in\mathbb{R}^n\setminus\{0\}} \sum c_i \sum \lambda_j c_j \delta_{ij}\end{aligned} \tag{6}$$

where δ_{ij} is the kronecker delta,

note that since the vectors are normalized, the squared sum of c_i is also normalized. the way to get the largest value is therefore for $c_L = 1$ for L corresponding to the index of the largest eigenvector

5. Using part 4, prove that for any unweighted, undirected graph the largest eigenvalue of its Laplacian L is at least the maximum degree of the graph G, in other words prove:

$$\lambda_{max}(L) \geq d_{max}(G)$$

(Hint: as always the quadratic form of the Laplacian should be used) **Bonus/Grads:** Show that actually we can do slightly better: $\lambda_{max}(L) \geq d_{max}(G) + 1$.

solution

consider the vector d_{max} is on the vertex with the highest degree, -1 on all elements adjacent to the vertex a value of -1, and 0 on all other vertecies. we should notice that every edge contributes $(d_{max}+1)^2$ to xLx and there are d_max edges.

the norm of this vector comes out to be $d_{max}(d_{max}+1)$ and thus the largest eigenvalue, which is always $\geq$ the quadratic form for any vector, must be greater than or equal to $d_{max}+1$

6. Let P_n be the path graph on n nodes. Prove that $\lambda_2(P_n) = O(\frac{1}{n^2})$.

(Hint: Here you should use the characterization for the second smallest eigenvalue of the Laplacian: $\lambda_2(L) = \min_{v\in\mathbb{R}^n\setminus\{0\}, v^T\cdot\mathbf{1}=0} \frac{v^TAv}{v^Tv}$, where $\mathbf{1}$ is the all-1s vector... One more hint: maybe the vector whose i^{th} coordinate is $n+1-2i$ will be helpful in the proof.)

solutionif we can find a single vector that has a quadratic form value that is $O(\frac{1}{n^2})$, then the second eigenvalue must be $\leq$ that vectors quadratic form value.

consider the vector described in the hint.

$$\frac{x^TLx}{x^Tx} = \frac{n-1}{\sum(n+1-2i)^2} \tag{7}$$

but notice the denominator is approximately $O(\sum_0^n n^2) = O(n^3)$ which means the whole expression is $O(\frac{1}{n^2})$

so thus the second eigenvalue must be $O(\frac{1}{n^2})$.

Ex.2 (20pts) (Two More Motifs) See Figure 1 that shows two more motif equations like the previous homeworks. Here, we use the adjacency matrix A and also the adjacency matrix C of the complement graph. We ask you to prove the two motif equations. (P_4: this is the path on 4 nodes, and the other patterns are as shown in the figure.)

$$\sum_{v \in V} (A^3 C)_{v,v} = 4(\#\diamondsuit) + 2(\#P_4) + 4(\#\triangleright\!\!-)$$

$$\sum_{(u,v) \in E} \binom{(AC)_{u,v}}{2} = (\#\triangleright\!\!-) + 3(\#\triangleright\!\!-)$$

Figure 1: Two extra Motif Equations. The matrix C corresponds to the complement graph.

solution

let us draw out all the possible ways the path is counted for each of the motifs in equation 1. let the red line represent no edge and the black lines represent edges

1. In a diamond, there are 4 ways to travel along 1 edge then 3 edges to return to the original vertex.

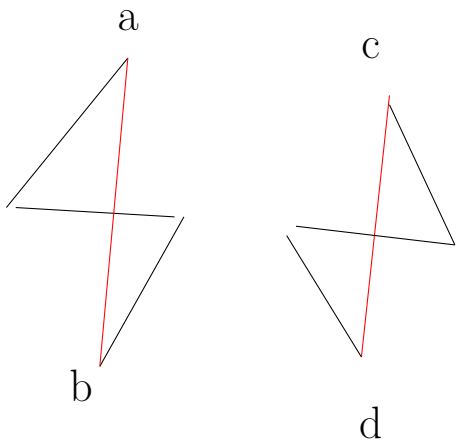


Figure 2: Four ways to count diamond, starting at nodes a, b, c, d

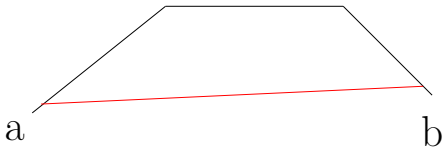


Figure 3: two ways to count the four path, staring at a, b

Figure 4: four ways to count tailed triangle starting at a, b, c, d

2. this motif counts: how many combinations of two are there to go from u to v given one non edge to one edge AND there being an edge from u to v.

   in the tailed triangle, there are exactly two ways for this to happen

   notice in figure 5, there are exactly two paths of length two, crosing one non edge (dashed), then 1 edge (solid) going from v to u. There are no other ways for this motif to be counted in the equation.

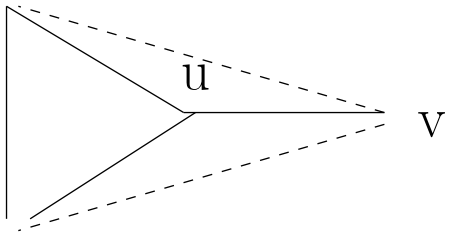


Figure 5: tailed triangle motif

in the 4 star graph there are 3 ways to choose two. where in the tailed triangle only 1 edge was a candidate (u, v), all three edges have two paths in the same way figure 5 depicts thus a 4 star graph is counted 3 times.

Ex.3 (20pts) (Embedding the Petersen graph) Check Figure 2 where we show the Petersen graph. We ask you the following:

- Compute the eigenvalues and eigenvectors of the Laplacian L matrix. Please comment on the symmetry observed.
- Do the spectral embedding of the Petersen graph onto the second and third smallest eigenvectors of L.
- Compute the second eigenvector and run the sweeping cut algorithm to split the graph into two pieces S and $V \setminus S$, so as to (try to) minimize the isoperimetric ratio. Output the set S, its boundary and its isoperimetric ratio. What is the optimum isoperimetric ratio of the graph?
- Choose any vertex that you want and delete two of its edges. Then repeat parts 1,2,3 for this graph.

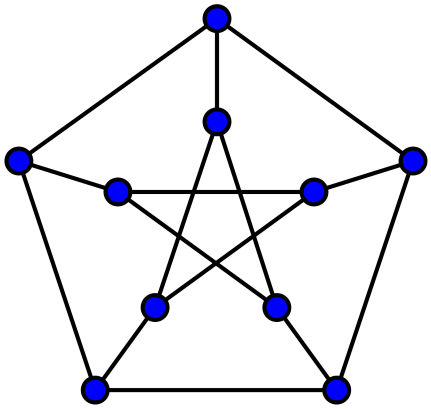


Figure 6: The Petersen Graph.

Ex.4 (30pts) (Interview Question I: Geometry and Embeddings)

Here we explore embeddings of random points. (feel free to use any software/programming language you want to do the calculations (e.g., Python or Matlab) and you don't have to include your calculations, just your final answers and plots.)

- Pick 100 random points in the unit square, by independently choosing their x and y coordinates uniformly at random from the interval [0,1] (there are many Python implementations for this). Form a graph by adding an edge between every pair of points whose Euclidean distance is at most $\frac{1}{4}$.

  Using the Laplacian, plot the embedding of the graph onto the second and third eigenvectors (those that correspond to the second and third smallest eigenvalues). Here do **not** overlay the edges of the graph just plot the vertices.

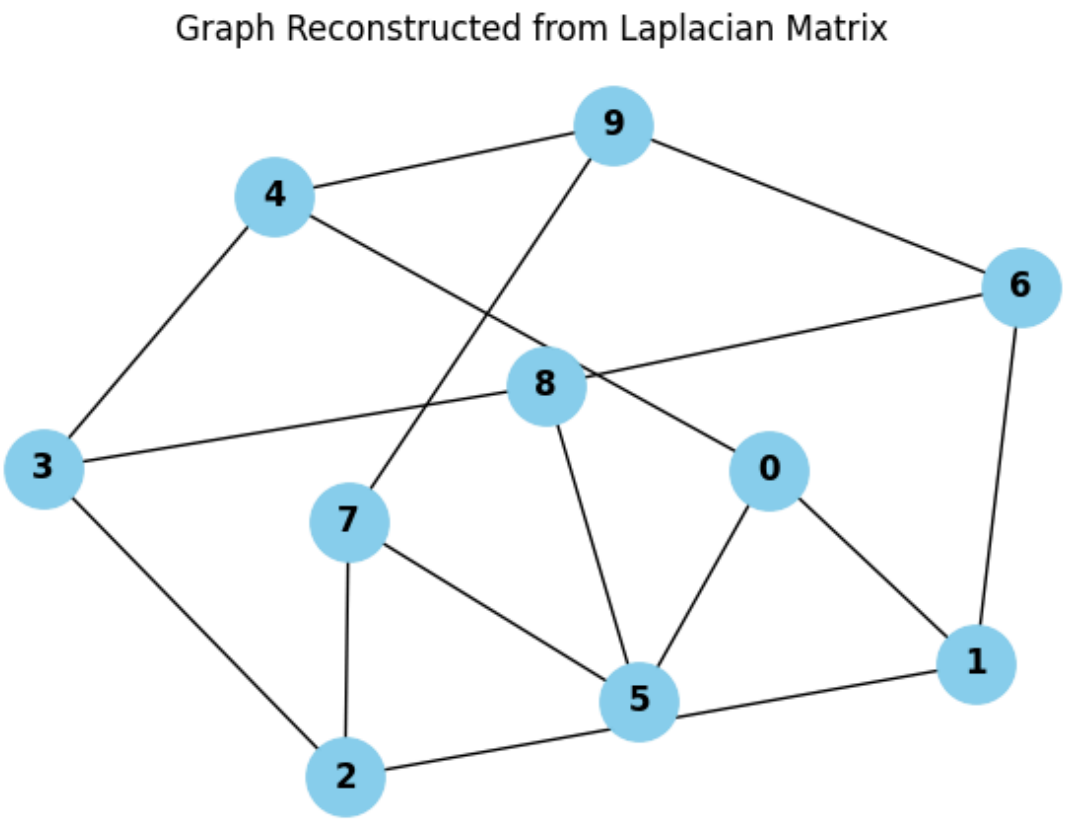
- Now, for all points in the original dataset whose x and y coordinates are **both less** than $\frac{1}{2}$, plot their images in the embedding in a different color. Are these points clustered together in the embedding? Why does this make sense?

EX 3

In [97]:
```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

L = nx.petersen_graph()
L:np.ndarray = nx.laplacian_matrix(L).toarray()
print(L)


A = -L.copy()
np.fill_diagonal(A, 0)


G = nx.from_numpy_array(A)

plt.figure(figsize=(6, 4))
nx.draw(G, with_labels=True, node_color='skyblue', node_size=800, font_weight='bold
plt.title("Graph Reconstructed from Laplacian Matrix")
```

```
[[ 3 -1  0  0 -1 -1  0  0  0  0]
 [-1  3 -1  0  0  0 -1  0  0  0]
 [ 0 -1  3 -1  0  0  0 -1  0  0]
 [ 0  0 -1  3 -1  0  0  0 -1  0]
 [-1  0  0 -1  3  0  0  0  0 -1]
 [-1  0  0  0  0  3  0 -1 -1  0]
 [ 0 -1  0  0  0  0  3  0 -1 -1]
 [ 0  0 -1  0  0 -1  0  3  0 -1]
 [ 0  0  0 -1  0 -1 -1  0  3  0]
 [ 0  0  0  0 -1  0 -1 -1  0  3]]
```

Out[97]: Text(0.5, 1.0, 'Graph Reconstructed from Laplacian Matrix')

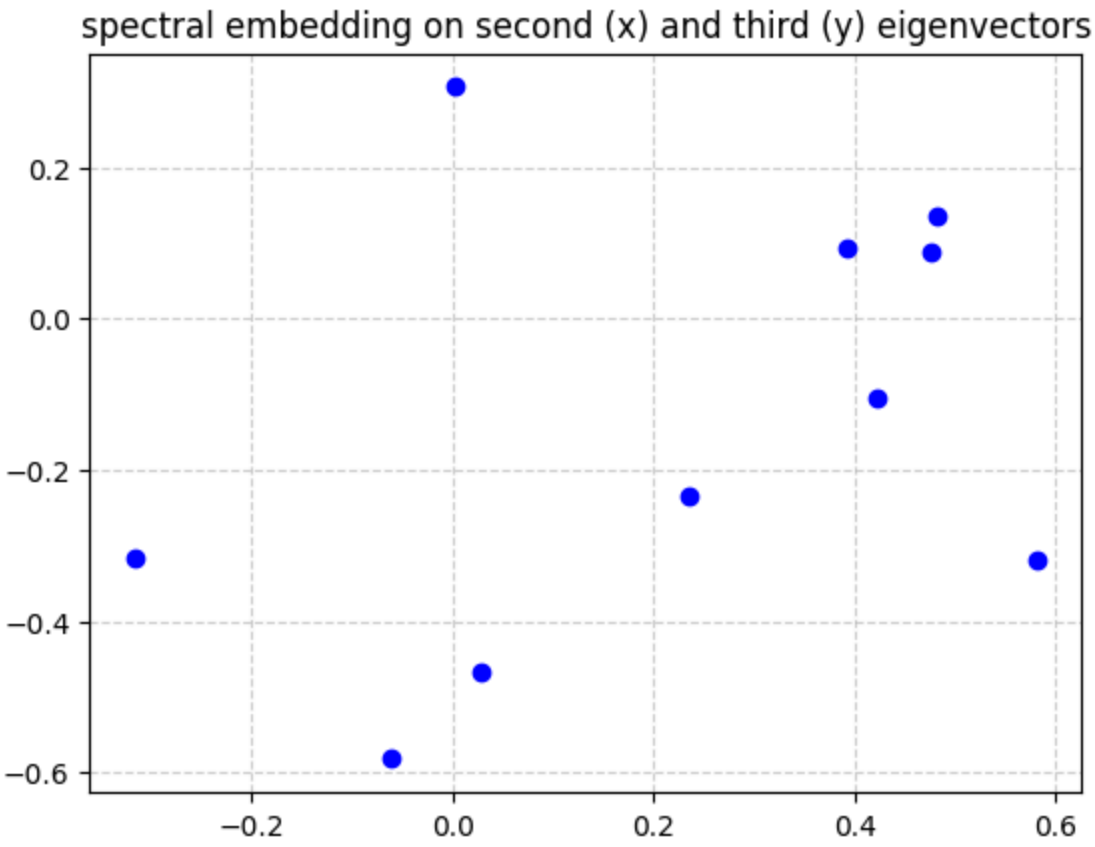


In [98]:
```
# print(np.linalg.eigvals(L))
np.set_printoptions(precision=4, suppress=True)
eigval, eigvec = np.linalg.eig(L)
print(eigval)
# print(sorted(np.round(eigval).astype(int).tolist()))
print(eigvec.T)
```

```
[ 5.  2. -0.  2.  5.  5.  5.  2.  2.  2.]
[[-0.6325  0.4216 -0.1054 -0.1054  0.4216  0.4216 -0.1054 -0.1054 -0.1054
  -0.1054]
 [ 0.7071  0.2357 -0.2357 -0.2357  0.2357  0.2357 -0.2357 -0.2357 -0.2357
  -0.2357]
 [-0.3162 -0.3162 -0.3162 -0.3162 -0.3162 -0.3162 -0.3162 -0.3162 -0.3162
  -0.3162]
 [ 0.0229  0.4766 -0.0959 -0.5263 -0.4153 -0.0384  0.5496 -0.0463 -0.0151
   0.0881]
 [-0.0007  0.0024  0.3956 -0.4894  0.0921 -0.0931 -0.3996 -0.3042  0.491
   0.3058]
 [-0.0316  0.4815 -0.5507  0.3046 -0.2038 -0.2145 -0.3807  0.3153  0.1453
   0.1346]
 [ 0.002  -0.0607 -0.1412 -0.18    0.38   -0.3233  0.2607  0.5232  0.1213
  -0.5819]
 [-0.0215  0.0279  0.436   0.0675 -0.422   0.3725 -0.3866  0.3406  0.0535
  -0.468 ]
 [-0.0224  0.5808  0.2281  0.0579 -0.2841 -0.3191  0.375  -0.4106  0.1139
  -0.3196]
 [-0.1508  0.3922  0.5525 -0.0043 -0.0609 -0.4821 -0.0095  0.1646 -0.4959
   0.0942]]
```

the first two vectors have a lot of the same values

In [99]:
```
second = 1
third = 9

x_val = eigvec[second]
y_val = eigvec[third]


plt.plot(x_val,y_val, color="blue",marker='o',linestyle='none')
# plt.plot(x_val[keep],y_val[keep], color="red",marker='o',linestyle='none')


plt.title("spectral embedding on second (x) and third (y) eigenvectors")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



In [100…
```
def sweeping_cut(L, v2):
    n = len(v2)
    sorted_indices = np.argsort(v2)

    best_ratio = float('inf')
    best_S = None
    best_boundary = None

    for i in range(1, n):
        S = sorted_indices[:i]
        V_minus_S = sorted_indices[i:]

        cut_submatrix = L[np.ix_(S, V_minus_S)]
```

```
        boundary_size = int(np.sum(cut_submatrix < 0))

        denominator = min(len(S), len(V_minus_S))
        ratio = boundary_size / denominator

        if ratio < best_ratio:
            best_ratio = ratio
            best_S = S.tolist()
            rows, cols = np.where(cut_submatrix < 0)
            best_boundary = [(S[r], V_minus_S[c]) for r, c in zip(rows, cols)]

    best_S = np.array(best_S).tolist()
    best_boundary = np.array(best_boundary).tolist()

    return best_S, best_boundary, best_ratio

S, boundary, ratio = sweeping_cut(L, eigvec[1])



print("\n--- Final Result ---")
print(f"Optimal Set S: {S}")
print(f"Boundary Edges: {boundary}")
print(f"Isoperimetric Ratio: {ratio}")
```

```
--- Final Result ---
Optimal Set S: [2, 6, 4, 7, 1, 9]
Boundary Edges: [[2, 3], [6, 8], [4, 0], [4, 3], [7, 5], [1, 0]]
Isoperimetric Ratio: 1.5
```

the isoperimetric ratio is 1.5 but we know we can do better ratio of 1. all the center nodes as S gives an isoperimetric ratio of $5/5 = 1$

In [101…

```
L[0][0] = 1
L[0][1] = 0
L[0][4] = 0
L[1][0] = 0
L[4][0] = 0

L[1][1] = 2
L[4][4]  =2

print(L)
```

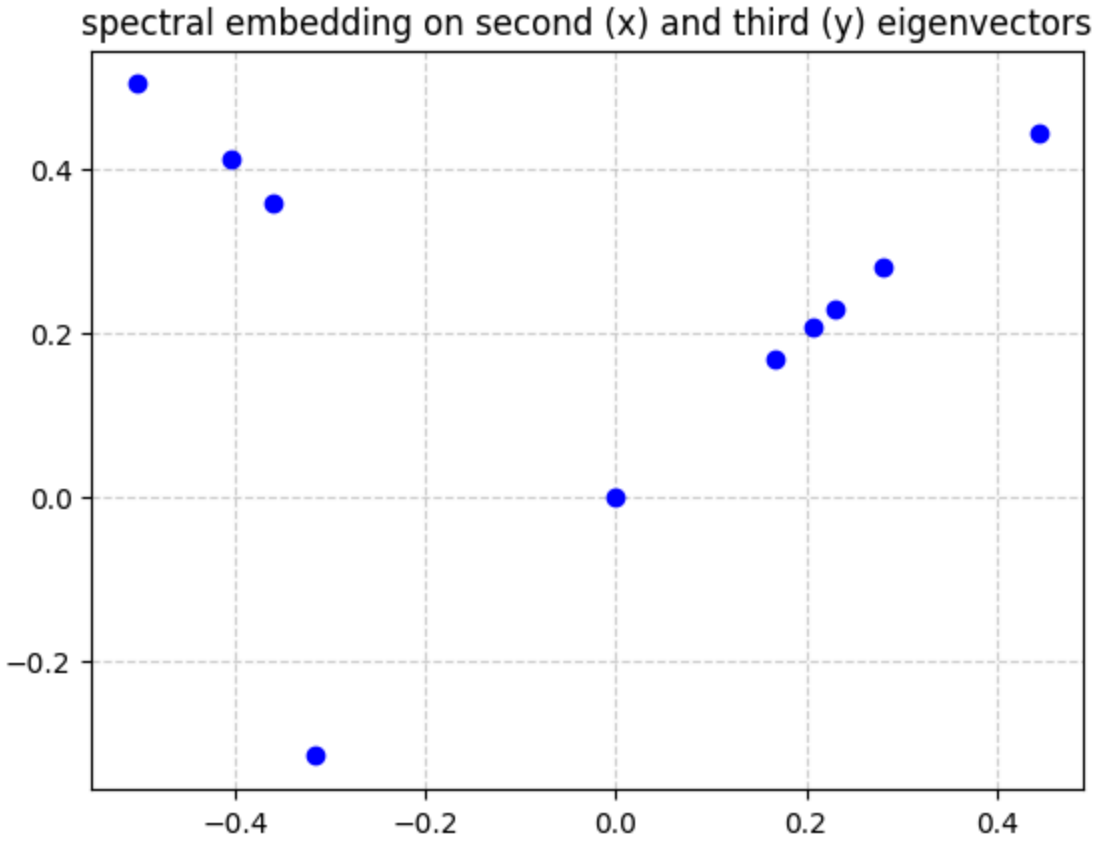
```
[[ 1  0  0  0  0 -1  0  0  0  0]
 [ 0  2 -1  0  0  0 -1  0  0  0]
 [ 0 -1  3 -1  0  0  0 -1  0  0]
 [ 0  0 -1  3 -1  0  0  0 -1  0]
 [ 0  0  0 -1  2  0  0  0  0 -1]
 [-1  0  0  0  0  3  0 -1 -1  0]
 [ 0 -1  0  0  0  0  3  0 -1 -1]
 [ 0  0 -1  0  0 -1  0  3  0 -1]
 [ 0  0  0 -1  0 -1 -1  0  3  0]
 [ 0  0  0  0 -1  0 -1 -1  0  3]]
```

In [102…
```
# print(np.linalg.eigvals(L))
np.set_printoptions(precision=4, suppress=True)
eigval, eigvec = np.linalg.eig(L)
print(eigval)
# print(sorted(np.round(eigval).astype(int).tolist()))
print(eigvec.T)
```

```
[5.     0.     0.5858 2.     3.4142 1.2679 2.     4.7321 5.     2.    ]
[[-0.138  -0.138   0.207   0.207  -0.138   0.5521  0.207  -0.483  -0.483
   0.207 ]
 [-0.3162 -0.3162 -0.3162 -0.3162 -0.3162 -0.3162 -0.3162 -0.3162 -0.3162
  -0.3162]
 [-0.8125  0.238   0.1683  0.1683  0.238  -0.3366  0.1683  0.      0.
   0.1683]
 [-0.4082 -0.4082 -0.     -0.     -0.4082  0.4082 -0.      0.4082  0.4082
  -0.    ]
 [ 0.2326 -0.3971  0.2808  0.2808 -0.3971 -0.5615  0.2808  0.     -0.
   0.2808]
 [ 0.      0.628   0.2299 -0.2299 -0.628  -0.      0.2299  0.     -0.
  -0.2299]
 [-0.0385 -0.0385 -0.3595  0.2087 -0.0385  0.0385  0.3595 -0.5296  0.6067
  -0.2087]
 [-0.     -0.3251  0.444  -0.444   0.3251  0.      0.444  -0.      0.
  -0.444 ]
 [-0.0031 -0.0031 -0.4034  0.4128 -0.0031  0.0125  0.4128  0.3972 -0.4191
  -0.4034]
 [ 0.0141  0.0141 -0.5034 -0.4959  0.0141 -0.0141  0.5034 -0.0216 -0.0065
   0.4959]]
```

much of the symmetry is lost. 4th eigenvector still has good symmetry, and final one also has a lot of repeated values

In [103…
```
second = 2
third = 6

x_val = eigvec[second]
y_val = eigvec[third]


plt.plot(x_val,y_val, color="blue",marker='o',linestyle='none')
# plt.plot(x_val[keep],y_val[keep], color="red",marker='o',linestyle='none')

plt.title("spectral embedding on second (x) and third (y) eigenvectors")


plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



In [104…

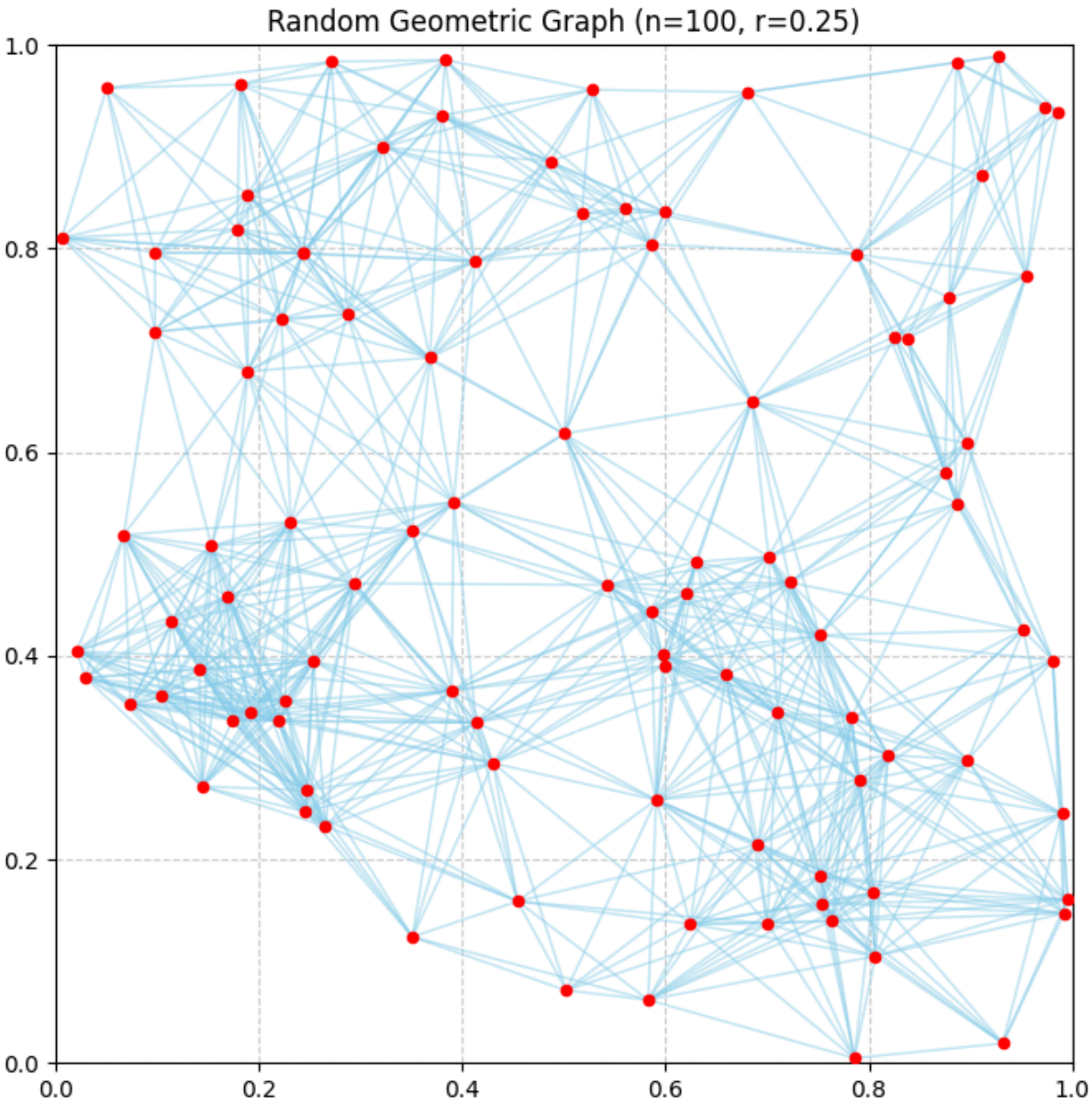
```
S, boundary, ratio = sweeping_cut(L, eigvec[2])



print("\n--- Final Result ---")
print(f"Optimal Set S: {S}")
print(f"Boundary Edges: {boundary}")
print(f"Isoperimetric Ratio: {ratio}")
```

```
--- Final Result ---
Optimal Set S: [9, 8, 6, 1, 3]
Boundary Edges: [[9, 4], [9, 7], [8, 5], [1, 2], [3, 2], [3, 4]]
Isoperimetric Ratio: 1.2
```

this is definitely not the optimal as choosing eigenvector 5 gives an isoperimetric ratio of 1. I think it can go even lower though.

EX 4

In [2]:
```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial.distance import pdist, squareform

n_points = 100
threshold = 0.25


coords = np.random.uniform(0, 1, size=(n_points, 2))


dist_matrix = squareform(pdist(coords))

A = (dist_matrix <= threshold) & (dist_matrix > 0)
degrees = np.sum(A, axis=1)
D = np.diag(degrees)
L = D - A

# --- Visualization ---
plt.figure(figsize=(8, 8))
plt.scatter(coords[:, 0], coords[:, 1], color='red', zorder=5, s=20)

# Plot the edges
for i in range(n_points):
    for j in range(i + 1, n_points):
        if A[i, j]:
            x_values = [coords[i, 0], coords[j, 0]]
            y_values = [coords[i, 1], coords[j, 1]]
            plt.plot(x_values, y_values, color='skyblue', alpha=0.5, lw=1)

plt.title(f"Random Geometric Graph (n=100, r={threshold})")
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.gca().set_aspect('equal', adjustable='box')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



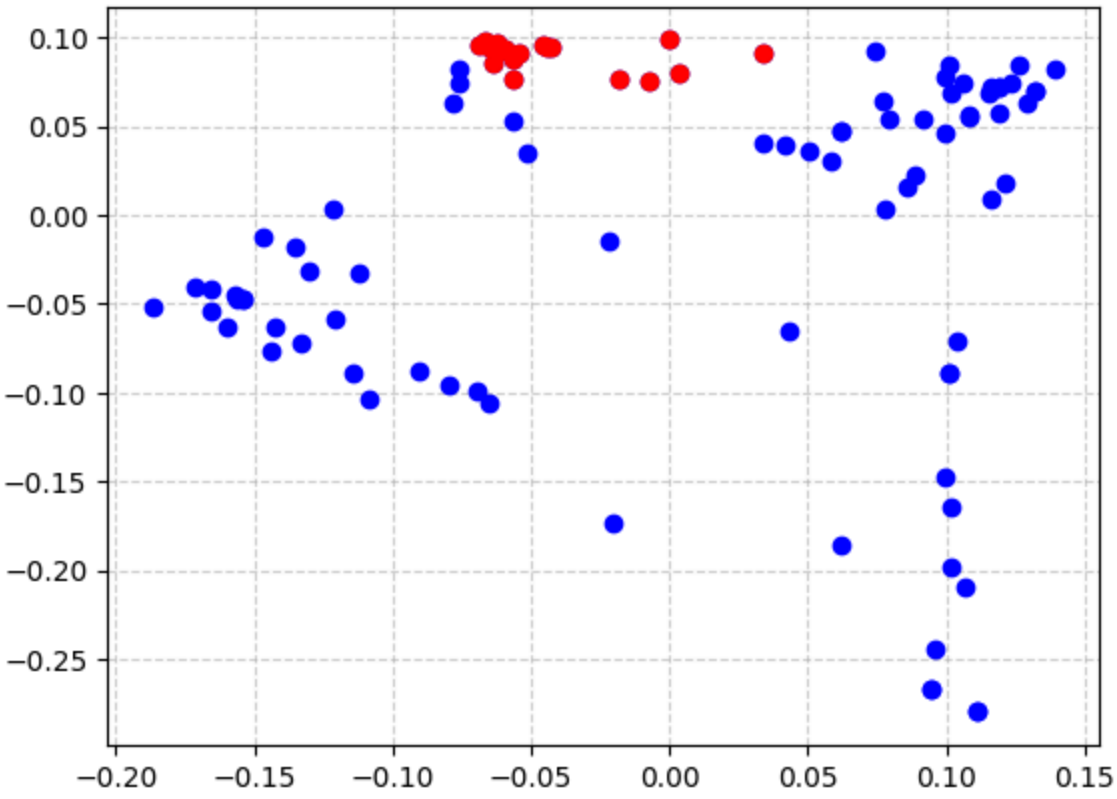
In [12]:
```
np.set_printoptions(precision=4, suppress=True)
eigval,eigvec = np.linalg.eig(L)
eigvec = eigvec.T
indexed_sorted = sorted(enumerate(eigval), key=lambda x: x[1])

# print(eigvec[0])
print(eigval)
```

```
[-0.      0.9927  1.1014  1.9341  4.3071  4.7249  5.9483  6.7604 27.2859
  7.483  26.3112 26.1185  8.1894  8.2816  9.3892  9.3977 25.5904  9.7432
 25.5339 25.3564 25.1408  8.      9.9908 10.4535 10.7652 10.8089 24.7715
  9.     24.4793 24.5163 24.1794 11.4544 11.7355 12.0611 23.7361 12.2398
 23.625  23.5616 12.4143 12.5596 22.9041 22.7275 12.7126 12.8062 13.0949
 13.4299 22.5346 24.     22.4292 13.5435 13.5744 13.7993 13.8508 14.1046
 15.147  14.8464 14.674  14.5905 14.3395 21.6761 21.6134 15.489  21.4923
 21.2162 15.9103 16.1654 20.8491 16.5759 20.7549 16.8007 20.6413 20.5326
 20.3158 20.2257 17.1162 19.4382 19.1796 18.7976 18.2619 18.235  18.5849
 17.3871 19.6243 17.9377 20.0143 17.6388 18.647  17.7378 17.2283 19.8747
 19.7834 17.5642 17.8143 19.8244 13.     14.     17.     20.     20.
 20.    ]
```

In [35]:
```
mask = (coords[:, 0] < 0.5) & (coords[:, 1] < 0.5)
keep = np.where(mask)[0]
print(keep)
```

```
[ 0  8 13 21 25 27 28 32 35 36 43 51 53 66 72 80 85 86 88 94 97 99]
```

In [36]:
```
# print(eigvec[indexed_sorted[1][0]],eigvec[indexed_sorted[2][0]])
# plt.plot(eigvec[indexed_sorted[1][0]],eigvec[indexed_sorted[2][0]], marker='o', l
x_val = eigvec[indexed_sorted[1][0]]
y_val = eigvec[indexed_sorted[2][0]]


plt.plot(x_val,y_val, color="blue",marker='o',linestyle='none')
plt.plot(x_val[keep],y_val[keep], color="red",marker='o',linestyle='none')



plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



since the points in the bottom quarter are all close and the second/third eigvectors try to keep everything close in value, they appear clustered together in the spectral embedding